SEMANA 7 · DESARROLLO FRONTEND

Control de Empleados

Frontend en Angular conectado a una API FastAPI

Manual paso a paso: instalación, estructura, servicios globales, mantenedor completo, pruebas y resultados.

Frontend	Angular 22 · TypeScript · Vitest
Backend	Python 3.11 · FastAPI · SQLite
Fecha	Noviembre de 2026

Contenido

1. Qué vamos a construir
2. Requisitos e instalación
3. El backend que usaremos
4. Crear el proyecto Angular
5. Estructura de carpetas
6. Configuración inicial
7. Etapa 1: Servicio de logs
8. Etapa 2: Servicio de alertas
9. Etapa 3: Servicio de carga (spinner)
10. Conexión con la API: ApiService
11. Modelos y servicios de datos
12. Etapa 4: Páginas del mantenedor
13. Pruebas y resultados
14. Problemas comunes
15. Cómo seguir ampliando el proyecto

1. Qué vamos a construir

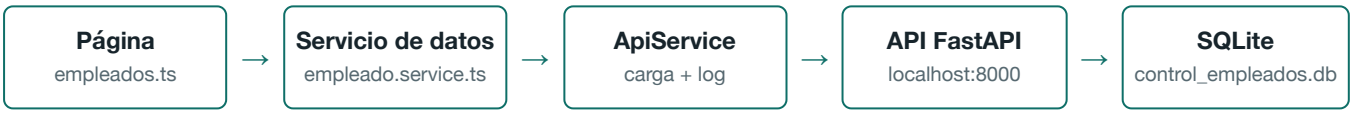
En la semana 7 se creó un backend con FastAPI que guarda empleados y conoce las **16 regiones, 56 provincias y 346 comunas de Chile**. En este manual construimos el **frontend en Angular** que usa esa API: una aplicación para administrar empleados, departamentos y cargos.

El trabajo se hizo por etapas. Cada etapa se construyó, se explicó y se probó antes de pasar a la siguiente:

Etapa	Qué se construye	Para qué sirve
1	Servicio de logs	Guardar los eventos y errores importantes de la aplicación
2	Servicio de alertas	Mostrar mensajes de éxito, error, advertencia e información
3	Servicio de carga	Oscurecer la pantalla y bloquearla mientras se espera a la API
4	Mantenedor	Listar, buscar, crear, editar y eliminar empleados, departamentos y cargos

Cómo viaja una petición

Cuando el usuario presiona **Guardar**, esto es lo que ocurre:



La respuesta vuelve por el mismo camino. Al final, la página muestra una **alerta** de éxito o de error.

2. Requisitos e instalación

Se necesitan tres programas instalados en el computador. Para saber si ya los tienes, abre una terminal y escribe:

```
Terminal
```

```
node -v       # Node.js 20 o superior (en este proyecto: v22)
npm -v        # se instala junto con Node.js
python3 -V    # Python 3.10 o superior (en este proyecto: 3.11)
```

- **Node.js**: se descarga desde nodejs.org (versión LTS). Trae `npm`, el instalador de paquetes de JavaScript.
- **Python 3**: se descarga desde python.org. Se usa para el backend.
- **Un editor**: Visual Studio Code, con la extensión *Angular Language Service*.

> No es necesario instalar Angular CLI de forma global. En este manual se usa con `npx`, que descarga la versión indicada solo para ese comando.

3. El backend que usaremos

El backend está en la carpeta `Semana 7/backend`. Antes de usar el frontend, la API tiene que estar encendida.

Terminal

```
cd "Semana 7/backend"
python3 -m venv venv              # crea el entorno virtual (solo la primera vez)
source venv/bin/activate          # en Windows: venv\Scripts\activate
pip install -r requirements.txt   # instala FastAPI y Uvicorn (solo la primera vez)
uvicorn main:app --reload         # enciende la API en http://localhost:8000
```

Al abrir `http://localhost:8000/docs` se ve la documentación interactiva de la API. Estas son las rutas que usa el frontend:

Método	Ruta	Uso en el frontend
GET	/regiones	Llenar el selector de región
GET	/regiones/{id}/provincias	Llenar provincias al elegir una región
GET	/provincias/{id}/comunas	Llenar comunas al elegir una provincia
GET	/comunas/{id}	Saber región y provincia de un empleado al editarlo
GET · POST · PUT · DELETE	/departamentos	Mantenedor de departamentos
GET · POST · PUT · DELETE	/cargos	Mantenedor de cargos
GET · POST · PUT · DELETE	/empleados	Mantenedor de empleados (GET acepta `?buscar=`)

La API responde con códigos que el frontend aprovecha para mostrar mensajes claros:

Código	Significado	Ejemplo
200 / 201 / 204	Todo salió bien	Empleado creado o eliminado
400	Una referencia no existe	La comuna indicada no existe
404	No se encontró	Departamento no encontrado
409	Conflicto	Ya existe un registro con ese rut
422	Datos inválidos	El dígito verificador del RUT no es correcto

CORS: la API solo acepta peticiones desde `http://localhost:4200`, que es la dirección donde corre Angular. Esto se configura en `backend/main.py`.

4. Crear el proyecto Angular

Desde la carpeta `Semana 7` se creó el proyecto con este comando:

```
Terminal
cd "Semana 7"
npx @angular/cli@22 new frontend --style=css --ssr=false --skip-git --routing --defaults --ai-config=none
```

Opción	Qué hace
`--style=css`	Usa CSS normal para los estilos
`--ssr=false`	No activa el renderizado en servidor (no lo necesitamos)
`--skip-git`	No crea un repositorio Git
`--routing`	Prepara las rutas para tener varias páginas
`--defaults`	Acepta las opciones por defecto sin preguntar
`--ai-config=none`	No agrega archivos de configuración para asistentes de IA

Después se entra a la carpeta del proyecto. **Todos los comandos de este manual se ejecutan desde aquí.**

```
Terminal
cd frontend
```

El archivo de configuración `src/environments/environment.ts` se crea a mano (en VS Code: clic derecho en `src` → *New Folder* → `environments`, y dentro *New File* → `environment.ts`). Su contenido está en la sección 6.

> **Ojo:** no uses `ng generate environments`. Ese comando crea además `environment.development.ts` vacío, y `npm start` usaría ese archivo: `apiUrl` quedaría sin valor y ninguna petición funcionaría.

Crear archivos con ng generate

Angular CLI crea los archivos con la estructura correcta. Así no hay que escribirlos desde cero ni equivocarse en los nombres. La forma general es:

```
Terminal
npx ng generate <tipo> <carpeta>/<nombre> [opciones]
```

Comando	Qué crea	Ejemplo de resultado
`npx ng g interface models/log --type=model`	Una interface (modelo)	`models/log.model.ts`
`npx ng g service services/ui/alerta --type=service`	Un servicio y su prueba	`alerta.service.ts` y `alerta.service.spec.ts`

Comando	Qué crea	Ejemplo de resultado
`npx ng g component components/menu --skip-tests`	Un componente	`menu.ts`, `menu.html`, `menu.css`

Opción	Qué hace
`g`	Abreviatura de `generate`
`--type=model` / `--type=service`	Agrega `.model` o `.service` al nombre del archivo, y `Service` al nombre de la clase (`AlertaService`)
`--skip-tests`	No crea el archivo de prueba `.spec.ts`
`--style=none`	No crea el archivo `.css` (las páginas usan `styles.css`)

Si la carpeta no existe, el comando la crea. Al ejecutarlo, la terminal muestra cada archivo creado:

Resultado

```
CREATE src/app/services/ui/alerta.service.spec.ts (356 bytes)
CREATE src/app/services/ui/alerta.service.ts (84 bytes)
```

Cada etapa de este manual comienza con los comandos que crean sus archivos. Después se reemplaza el contenido generado por el código que se muestra en esa sección.

Comandos que se usan durante todo el desarrollo:

Comando	Para qué sirve
`npm start`	Levanta la aplicación en http://localhost:4200 y se recarga sola al guardar cambios
`npx ng test --watch=false`	Ejecuta las pruebas unitarias una vez
`npx ng build`	Genera la versión final en la carpeta `dist/`

5. Estructura de carpetas

Cada tipo de archivo vive en su propia carpeta. Así es fácil encontrar las cosas y el proyecto puede crecer sin desordenarse.

Estructura de src/

```
src/
├── environments/
│   └── environment.ts          Dirección de la API
├── styles.css                  Estilos comunes a todas las páginas
└── app/
    ├── models/                 Forma de los datos (interfaces)
    │   ├── alerta.model.ts
    │   ├── cargo.model.ts
    │   ├── departamento.model.ts
    │   ├── empleado.model.ts
    │   ├── log.model.ts
    │   └── territorio.model.ts
    ├── services/
    │   ├── ui/                 Lo que se ve en pantalla
    │   │   ├── alerta.service.ts
    │   │   └── carga.service.ts
    │   ├── sistema/            Funcionamiento interno
    │   │   ├── log.service.ts
    │   │   └── error-global.service.ts
    │   └── api/                Comunicación con el backend
    │       ├── api.service.ts
    │       ├── mensaje-error.ts
    │       ├── empleado.service.ts
    │       ├── departamento.service.ts
    │       ├── cargo.service.ts
    │       └── territorio.service.ts
    ├── components/             Piezas que se repiten en toda la app
    │   ├── menu/
    │   ├── alertas/
    │   └── carga/
    ├── pages/                  Una carpeta por pantalla
    │   ├── empleados/
    │   ├── departamentos/
    │   ├── cargos/
    │   └── logs/
    ├── app.ts / app.html       Componente principal
    ├── app.config.ts           Configuración global
    └── app.routes.ts           Rutas (qué página va en cada dirección)
```

Carpeta	Qué guarda	Regla
models	Interfaces de TypeScript	Solo describen datos, no tienen lógica
services	Clases con lógica reutilizable	No tienen HTML. Se usan con `inject()`
components	Piezas visuales comunes	Van una sola vez en `app.html`
pages	Pantallas completas	Cada una tiene su ruta en `app.routes.ts`

Cada componente y cada página tiene su **.ts** (lógica) y su **.html** (vista). Los componentes además tienen su **.css**; las páginas usan los estilos comunes de `styles.css`. Los archivos **.spec.ts** que están junto a algunos servicios son sus pruebas.

Todos los comandos juntos

Esta es la lista completa, en el orden en que se usa en el manual. Cada etapa repite los suyos.

Terminal (carpeta frontend)

```
# Etapa 1: logs
npx ng g interface models/log --type=model
npx ng g service services/sistema/log --type=service
npx ng g service services/sistema/error-global --type=service --skip-tests
npx ng g component pages/logs --skip-tests --style=none

# Etapa 2: alertas
npx ng g interface models/alerta --type=model
npx ng g service services/ui/alerta --type=service
npx ng g component components/alertas --skip-tests

# Etapa 3: carga
npx ng g service services/ui/carga --type=service
npx ng g component components/carga --skip-tests

# Conexión con la API
npx ng g service services/api/api --type=service

# Modelos y servicios de datos
npx ng g interface models/departamento --type=model
npx ng g interface models/cargo --type=model
npx ng g interface models/empleado --type=model
npx ng g interface models/territorio --type=model
npx ng g service services/api/departamento --type=service --skip-tests
npx ng g service services/api/cargo --type=service --skip-tests
npx ng g service services/api/empleado --type=service --skip-tests
npx ng g service services/api/territorio --type=service --skip-tests

# Etapa 4: menú y páginas
npx ng g component components/menu --skip-tests
npx ng g component pages/departamentos --skip-tests --style=none
npx ng g component pages/cargos --skip-tests --style=none
npx ng g component pages/empleados --skip-tests --style=none
```

`mensaje-error.ts` y `mensaje-error.spec.ts` se crean a mano, porque contienen una función y no una clase de Angular.

6. Configuración inicial

environment.ts

Guarda la dirección de la API en un solo lugar. Si la API cambia de puerto, se modifica solo aquí.

src/environments/environment.ts

```
// Configuración general de la aplicación.
export const environment = {
  // Dirección de la API de FastAPI.
  apiUrl: 'http://localhost:8000',
};
```

app.config.ts

Registra lo que Angular pone a disposición de toda la aplicación:

- `provideHttpClient()` habilita las peticiones a la API. Sin esto aparece el error *NullInjectorError: No provider for HttpClient*.
- `ErrorHandler` se reemplaza por nuestro `ErrorGlobalService` (Etapa 1).
- `provideRouter(routes)` activa las rutas.

src/app/app.config.ts

```
// ============================================================
// app.config.ts
// Configuración global de la aplicación.
// ============================================================
import { provideHttpClient } from '@angular/common/http';
import { ApplicationConfig, ErrorHandler, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { ErrorGlobalService } from './services/sistema/error-global.service';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    // Los errores no controlados pasan por nuestro servicio.
    { provide: ErrorHandler, useClass: ErrorGlobalService },
    provideRouter(routes),
    // Habilita HttpClient, necesario para hablar con la API.
    provideHttpClient(),
  ],
};
```

app.routes.ts

Indica qué página se muestra en cada dirección. La dirección vacía redirige a `/empleados`.

src/app/app.routes.ts

```
// ============================================================
// app.routes.ts
// Indica qué página se muestra en cada dirección.
// ============================================================
import { Routes } from '@angular/router';
import { Cargos } from './pages/cargos/cargos';
import { Departamentos } from './pages/departamentos/departamentos';
```

```
import { Empleados } from './pages/empleados/empleados';
import { Logs } from './pages/logs/logs';

export const routes: Routes = [
  { path: '', redirectTo: 'empleados', pathMatch: 'full' },
  { path: 'empleados', component: Empleados },
  { path: 'departamentos', component: Departamentos },
  { path: 'cargos', component: Cargos },
  { path: 'logs', component: Logs },
];
```

7. Etapa 1: Servicio de logs

Un **log** es un registro de lo que pasa en la aplicación: qué se guardó, qué petición falló y cuándo. En lugar de escribir `console.log` por todos lados, la aplicación usa un único servicio. Así todos los eventos quedan en el mismo lugar y se pueden ver en la página **Registro de eventos**.

Crear los archivos

Terminal (carpeta frontend)

```
npx ng g interface models/log --type=model
npx ng g service services/sistema/log --type=service
npx ng g service services/sistema/error-global --type=service --skip-tests
npx ng g component pages/logs --skip-tests --style=none
```

Crea el modelo, el servicio de logs con su prueba, el servicio de errores globales y la página **logs**. Después se reemplaza el contenido de cada archivo por el código de esta sección.

El modelo

src/app/models/log.model.ts

```
// Forma de un registro del log.
export interface Log {
  fecha: Date;
  nivel: 'info' | 'advertencia' | 'error';
  mensaje: string;
}
```

`nivel` solo acepta tres valores. Si alguien escribe otro, TypeScript marca el error antes de ejecutar.

El servicio

src/app/services/sistema/log.service.ts

```
// ============================================================
// log.service.ts
// Guarda los eventos importantes de la aplicación.
// Uso: this.log.info('Empleado creado');
// ============================================================
import { Injectable, signal } from '@angular/core';
import { Log } from '../../models/log.model';

@Injectable({ providedIn: 'root' })
export class LogService {
  // Lista de registros. La página "Registro de eventos" la muestra.
  logs = signal<Log[]>([]);

  info(mensaje: string): void {
    this.registrar('info', mensaje);
  }

  advertencia(mensaje: string): void {
    this.registrar('advertencia', mensaje);
  }

  error(mensaje: string): void {
    this.registrar('error', mensaje);
```

```
  }

  limpiar(): void {
    this.logs.set([]);
  }

  private registrar(nivel: Log['nivel'], mensaje: string): void {
    // El más nuevo queda primero. slice guarda solo los últimos 100.
    this.logs.update((lista) => [{ fecha: new Date(), nivel, mensaje }, ...lista].slice(0, 100));
    console.log(`[${nivel}] ${mensaje}`);
  }
}
```

Cómo funciona

1. `@Injectable({ providedIn: 'root' })` hace que Angular cree **una sola instancia** para toda la aplicación. Todas las páginas comparten los mismos logs.
2. `logs` es un **signal**: un valor que, al cambiar, actualiza solo las pantallas que lo muestran.
3. `update()` agrega el registro nuevo al comienzo, y `slice(0, 100)` guarda solo los últimos 100 para no llenar la memoria.
4. Cada registro también se escribe en la consola del navegador (F12).

Cómo se usa

Desde cualquier componente o servicio

```
private log = inject(LogService);

this.log.info('Empleado guardado: Camila Núñez');
this.log.error('No se pudo cargar la lista');
```

Errores no controlados

A veces el código falla en un lugar donde nadie lo esperaba. Angular envía esos errores a una clase llamada `ErrorHandler`. La reemplazamos por la nuestra para que el error quede en el log y el usuario reciba un aviso.

src/app/services/sistema/error-global.service.ts

```
// ============================================================
// error-global.service.ts
// Reemplaza el manejador de errores de Angular. Todo error que
// nadie capturó llega aquí: se guarda en el log y se avisa.
// Se activa en app.config.ts.
// ============================================================
import { ErrorHandler, Injectable, inject } from '@angular/core';
import { AlertaService } from '../ui/alerta.service';
import { LogService } from './log.service';

@Injectable()
export class ErrorGlobalService implements ErrorHandler {
  private log = inject(LogService);
  private alerta = inject(AlertaService);

  handleError(error: Error): void {
    this.log.error(`Error no controlado: ${error.message}`);
    this.alerta.error('Ocurrió un error inesperado.');
  }
}
```

Se activa en `app.config.ts` con `{ provide: ErrorHandler, useClass: ErrorGlobalService }`.

La página Registro de eventos

src/app/pages/logs/logs.ts

```
// Página que muestra los eventos guardados en LogService.
import { DatePipe } from '@angular/common';
import { Component, inject } from '@angular/core';
import { LogService } from '../../services/sistema/log.service';

@Component({
  selector: 'app-logs',
  imports: [DatePipe],
  templateUrl: './logs.html',
})
export class Logs {
  servicio = inject(LogService);
}
```

src/app/pages/logs/logs.html

```
<h1>Registro de eventos</h1>

<button (click)="servicio.limpiar()">Limpiar</button>

<div class="tabla">
  <table>
    <thead>
      <tr>
        <th>Hora</th>
        <th>Nivel</th>
        <th>Mensaje</th>
      </tr>
    </thead>
    <tbody>
      @for (log of servicio.logs(); track $index) {
        <tr>
          <td>{{ log.fecha | date: 'HH:mm:ss' }}</td>
          <td>{{ log.nivel }}</td>
          <td>{{ log.mensaje }}</td>
        </tr>
      } @empty {
        <tr>
          <td colspan="3">No hay eventos.</td>
        </tr>
      }
    </tbody>
  </table>
</div>
```

`@for` recorre la lista y `@empty` muestra un mensaje cuando no hay datos. `date: 'HH:mm:ss'` es un **pipe**: transforma la fecha en un texto con la hora.

8. Etapa 2: Servicio de alertas

Las alertas son **mensajes en pantalla** para el usuario: “Empleado guardado”, “No se pudo conectar”. El servicio solo guarda los mensajes; un componente aparte se encarga de dibujarlos. Así cualquier parte de la aplicación puede mostrar una alerta sin tocar HTML.

Crear los archivos

Terminal (carpeta frontend)

```
npx ng g interface models/alerta --type=model
npx ng g service services/ui/alerta --type=service
npx ng g component components/alertas --skip-tests
```

El modelo

src/app/models/alerta.model.ts

```
// Forma de una alerta en pantalla.
export interface Alerta {
  id: number;
  tipo: 'exito' | 'error' | 'advertencia' | 'info';
  mensaje: string;
}
```

El servicio

src/app/services/ui/alerta.service.ts

```
// ============================================================
// alerta.service.ts
// Muestra mensajes en pantalla desde cualquier parte de la app.
// Uso: this.alerta.exito('Empleado guardado');
// El componente <app-alertas> se encarga de dibujarlos.
// ============================================================
import { Injectable, signal } from '@angular/core';
import { Alerta } from '../../models/alerta.model';

@Injectable({ providedIn: 'root' })
export class AlertaService {
  alertas = signal<Alerta[]>([]);
  private siguienteId = 1;

  exito(mensaje: string): void {
    this.mostrar('exito', mensaje);
  }

  error(mensaje: string): void {
    this.mostrar('error', mensaje);
  }

  advertencia(mensaje: string): void {
    this.mostrar('advertencia', mensaje);
  }

  info(mensaje: string): void {
    this.mostrar('info', mensaje);
  }

  cerrar(id: number): void {
```

```
    this.alertas.update((lista) => lista.filter((alerta) => alerta.id !== id));
  }

  private mostrar(tipo: Alerta['tipo'], mensaje: string): void {
    // Si el mismo mensaje ya está en pantalla, no se repite.
    if (this.alertas().some((alerta) => alerta.mensaje === mensaje)) {
      return;
    }

    const id = this.siguienteId++;
    this.alertas.update((lista) => [...lista, { id, tipo, mensaje }]);
    // Después de 5 segundos se cierra sola.
    setTimeout(() => this.cerrar(id), 5000);
  }
}
```

Cómo funciona

1. Hay un método por cada tipo: `exito`, `error`, `advertencia` e `info`. Los cuatro llaman a `mostrar()`.
2. Si el mismo mensaje ya está en pantalla, no se repite. Esto evita, por ejemplo, cuatro avisos iguales cuando la API está apagada.
3. Cada alerta recibe un `id` para poder cerrarla.
4. `setTimeout` la cierra sola después de 5 segundos. También se puede cerrar con la ×.

El componente que las dibuja

src/app/components/alertas/alertas.ts

```
// Dibuja las alertas guardadas en AlertaService.
import { Component, inject } from '@angular/core';
import { AlertaService } from '../../services/ui/alerta.service';

@Component({
  selector: 'app-alertas',
  templateUrl: './alertas.html',
  styleUrl: './alertas.css',
})
export class Alertas {
  servicio = inject(AlertaService);
}
```

src/app/components/alertas/alertas.html

```
<div class="alertas">
  @for (alerta of servicio.alertas(); track alerta.id) {
    <div [class]="'alerta ' + alerta.tipo" role="alert">
      <span>{{ alerta.mensaje }}</span>
      <button type="button" aria-label="Cerrar" (click)="servicio.cerrar(alerta.id)">×</button>
    </div>
  }
</div>
```

src/app/components/alertas/alertas.css

```
/* z-index mayor que la pantalla de carga: las alertas siempre se ven. */
.alertas {
  position: fixed;
  top: 16px;
  right: 16px;
  z-index: 1100;
  display: grid;
  gap: 10px;
```

```
  width: min(360px, calc(100vw - 32px));
}

.alerta {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  padding: 12px 14px;
  border-radius: 6px;
  color: #fff;
  box-shadow: 0 6px 18px rgb(0 0 0 / 20%);
}

.exito {
  background: #1b7a43;
}

.error {
  background: #b3261e;
}

.advertencia {
  background: #9a6200;
}

.info {
  background: #1d63a8;
}

button {
  border: none;
  background: transparent;
  color: #fff;
  font-size: 1.2rem;
  padding: 0 4px;
}
```

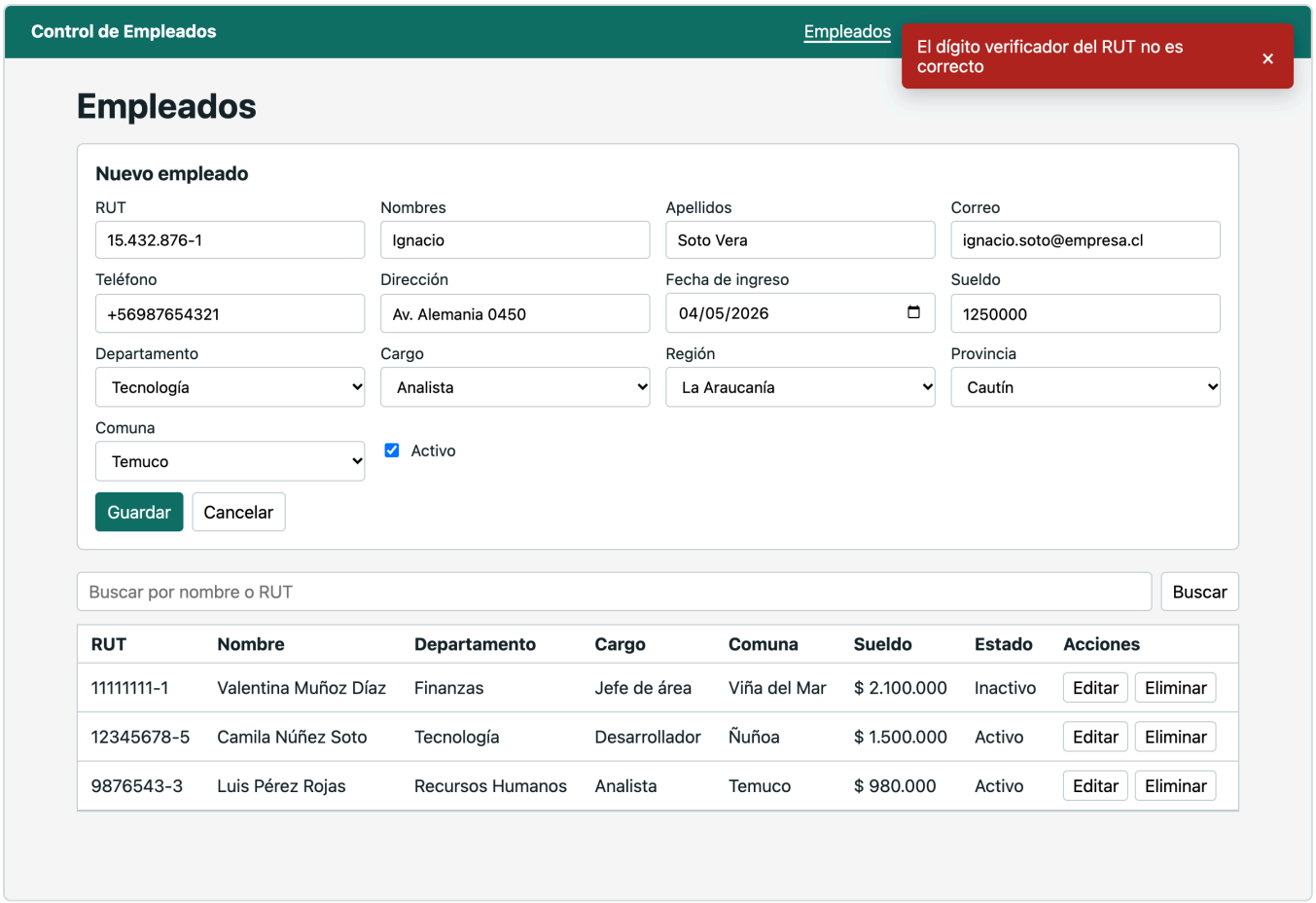
`position: fixed` deja las alertas siempre arriba a la derecha, aunque la página se desplace. `[class]="'alerta ' + alerta.tipo"` agrega la clase del tipo (`exito`, `error`...), y cada clase tiene su color.

Cómo se usa

Desde cualquier componente

```
private alerta = inject(AlertaService);

this.alerta.exito('Empleado guardado correctamente.');
this.alerta.error('No se pudo guardar.');
```



Alerta de error: la API rechazó un RUT con dígito verificador incorrecto.

9. Etapa 3: Servicio de carga (spinner)

Mientras se espera la respuesta de la API, la pantalla se oscurece, aparece un indicador giratorio y el usuario no puede interactuar. Así se evita, por ejemplo, que presione **Guardar** dos veces.

Crear los archivos

Terminal (carpeta frontend)

```
npx ng g service services/ui/carga --type=service
npx ng g component components/carga --skip-tests
```

El servicio

src/app/services/ui/carga.service.ts

```
// ============================================================
// carga.service.ts
// Controla la pantalla de carga (capa oscura con spinner).
// Usa un contador: con 3 peticiones a la vez, la pantalla sigue
// bloqueada hasta que termine la última.
// ============================================================
import { Injectable, computed, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class CargaService {
  private peticiones = signal(0);

  // true mientras exista al menos una petición en curso.
  activa = computed(() => this.peticiones() > 0);

  mostrar(): void {
    this.peticiones.update((n) => n + 1);
  }

  ocultar(): void {
    this.peticiones.update((n) => n - 1);
  }
}
```

¿Por qué un contador y no true/false?

La página de empleados hace 4 peticiones al abrirse. Si se usara `true/false`, la primera en terminar pondría `false` y quitaría el spinner aunque las otras 3 siguieran esperando. Con un contador, cada petición suma 1 al empezar y resta 1 al terminar. El spinner se va recién cuando el contador vuelve a 0.

`computed` crea un signal que se calcula a partir de otro: `activa` vale `true` mientras `peticiones` sea mayor que 0.

El componente

src/app/components/carga/carga.ts

```
// Capa oscura con spinner. Se muestra según CargaService.
import { Component, inject } from '@angular/core';
import { CargaService } from '../../services/ui/carga.service';

@Component({
```

```
  selector: 'app-carga',
  templateUrl: './carga.html',
  styleUrl: './carga.css',
})
export class Carga {
  servicio = inject(CargaService);
}
```

src/app/components/carga/carga.html

```
@if (servicio.activa()) {
  <div class="capa">
    <div class="spinner"></div>
    <p>Cargando...</p>
  </div>
}
```

src/app/components/carga/carga.css

```
/* La capa cubre toda la pantalla. Se oscurece recién a los 150 ms,
   así las respuestas rápidas no hacen parpadear la pantalla. */
.capa {
  position: fixed;
  inset: 0;
  z-index: 1000;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 12px;
  background: rgb(0 0 0 / 50%);
  color: #fff;
  animation: aparecer 120ms 150ms both;
}

.spinner {
  width: 48px;
  height: 48px;
  border: 5px solid rgb(255 255 255 / 30%);
  border-top-color: #fff;
  border-radius: 50%;
  animation: girar 800ms linear infinite;
}

@keyframes aparecer {
  from {
    opacity: 0;
  }
}

@keyframes girar {
  to {
    transform: rotate(360deg);
  }
}
```

- `position: fixed; inset: 0` hace que la capa cubra toda la pantalla y reciba los clics.
- `z-index: 1000` la pone sobre la página. Las alertas tienen 1100, para que se sigan viendo encima.
- La animación `aparecer` oscurece la pantalla recién a los 150 ms. Si la API responde rápido, la pantalla no parpadea.

Bloquear también el teclado

Una capa encima detiene el mouse, pero con **Tab** y **Enter** todavía se podrían presionar los botones de atrás. Por eso `app.html` agrega el atributo `inert` al contenido mientras la carga está activa. Un elemento `inert` no recibe clics ni foco.

src/app/app.html

```
<!-- inert bloquea clics y teclado mientras la pantalla está cargando. -->
<div [attr.inert]="carga.activa() ? '' : null">
  <app-menu />

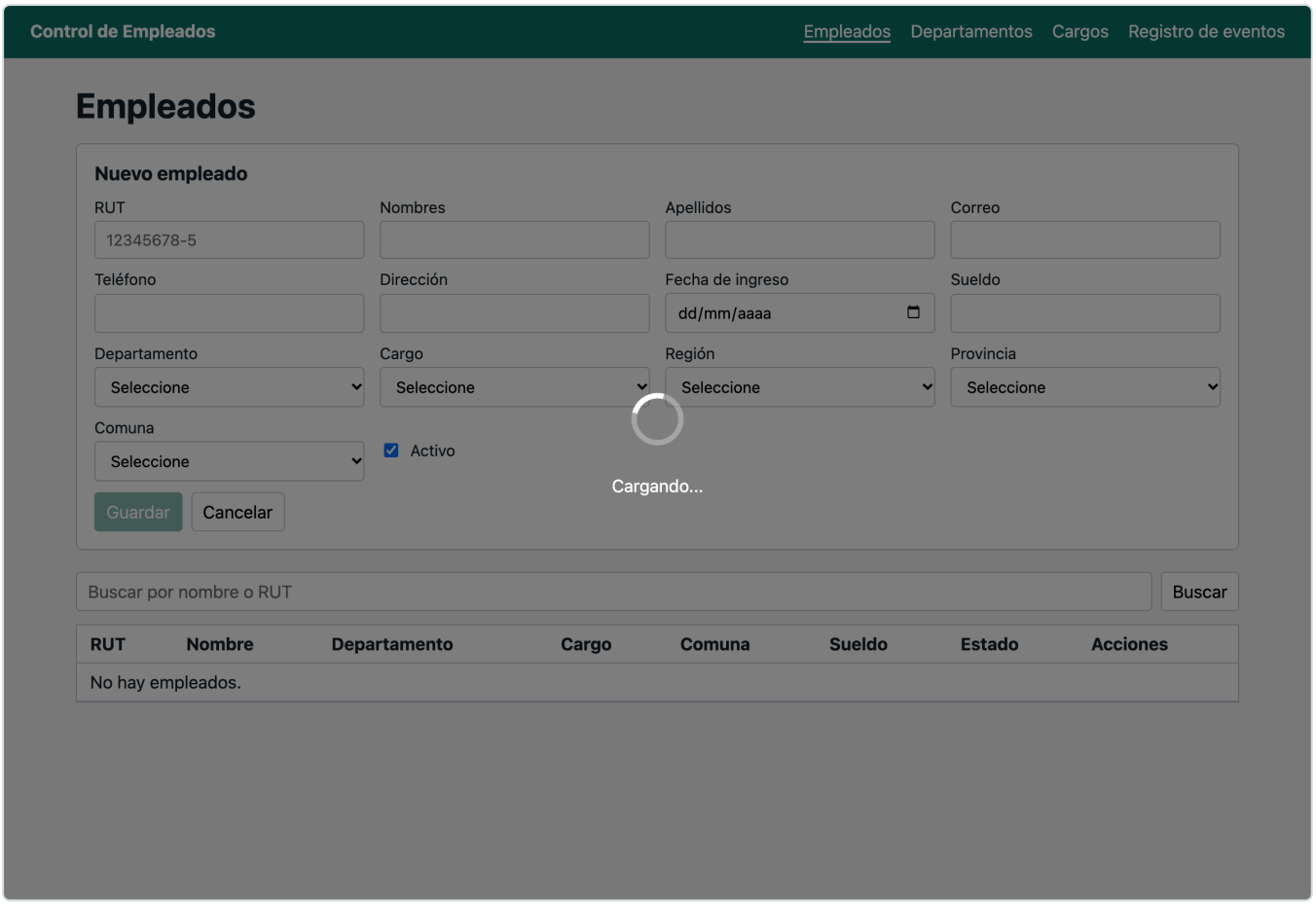
  <main class="contenido">
    <router-outlet />
  </main>
</div>

<app-carga />
<app-alertas />
```

src/app/app.ts

```
// ============================================================
// app.ts
// Componente principal: contiene lo común a todas las páginas.
// ============================================================
import { Component, inject } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Alertas } from './components/alertas/alertas';
import { Carga } from './components/carga/carga';
import { Menu } from './components/menu/menu';
import { CargaService } from './services/ui/carga.service';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Menu, Alertas, Carga],
  templateUrl: './app.html',
})
export class App {
  carga = inject(CargaService);
}
```



Pantalla de carga: la página queda oscurecida y bloqueada mientras llega la respuesta.

10. Conexión con la API: ApiService

Todas las peticiones pasan por un único servicio, `ApiService`. Por eso **todas** muestran el spinner y quedan registradas en el log, sin repetir ese código en cada página.

Crear los archivos

Terminal (carpeta frontend)

```
npx ng g service services/api/api --type=service
```

El archivo `mensaje-error.ts` (y su prueba `mensaje-error.spec.ts`) se crea a mano en la misma carpeta `services/api`, porque es una función y no un servicio.

El servicio

src/app/services/api/api.service.ts

```
// ============================================================
// api.service.ts
// Único lugar que habla con la API. Cada petición pasa por aquí,
// y por eso todas muestran la pantalla de carga y quedan en el log.
// Los demás servicios de esta carpeta lo usan así:
//   this.api.get<Cargo[]>('/cargos')
// ============================================================
import { HttpClient } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { Observable, finalize, tap } from 'rxjs';
import { environment } from '../../../environments/environment';
import { LogService } from '../sistema/log.service';
import { CargaService } from '../ui/carga.service';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  private carga = inject(CargaService);
  private log = inject(LogService);
  private url = environment.apiUrl;

  get<T>(ruta: string, params: Record<string, string> = {}): Observable<T> {
    return this.ejecutar(`GET ${ruta}`, this.http.get<T>(this.url + ruta, { params }));
  }

  post<T>(ruta: string, datos: unknown): Observable<T> {
    return this.ejecutar(`POST ${ruta}`, this.http.post<T>(this.url + ruta, datos));
  }

  put<T>(ruta: string, datos: unknown): Observable<T> {
    return this.ejecutar(`PUT ${ruta}`, this.http.put<T>(this.url + ruta, datos));
  }

  delete<T>(ruta: string): Observable<T> {
    return this.ejecutar(`DELETE ${ruta}`, this.http.delete<T>(this.url + ruta));
  }

  private ejecutar<T>(descripcion: string, peticion: Observable<T>): Observable<T> {
    this.carga.mostrar();

    return peticion.pipe(
      // tap permite mirar el resultado sin modificarlo.
      tap({
```

```
        next: () => this.log.info(`${descripcion}: correcto`),
        error: (error) => this.log.error(`${descripcion}: error ${error.status}`),
      }),
      // finalize se ejecuta siempre: si la petición sale bien o si falla.
      finalize(() => this.carga.ocultar()),
    );
  }
}
```

Cómo funciona

1. `get`, `post`, `put` y `delete` arman la URL completa y llaman a `ejecutar()`.
2. `ejecutar()` enciende la carga con `this.carga.mostrar()`.
3. `pipe()` agrega pasos a la respuesta. `tap` la mira sin cambiarla y guarda en el log si salió bien o mal.
4. `finalize` se ejecuta **siempre**, salga bien o mal, y apaga la carga. Así la pantalla nunca queda bloqueada por un error.

> **Regla del proyecto:** toda petición a la API debe pasar por `ApiService`. Si un servicio usa `HttpClient` directamente, esa petición no tendrá spinner ni log.

Mensajes de error entendibles

Cuando la API responde con un error, esta función lo convierte en un texto para el usuario:

`src/app/services/api/mensaje-error.ts`

```
// ============================================================
// mensaje-error.ts
// Convierte un error de la API en un texto para el usuario.
// Uso: this.alerta.error(mensajeError(error));
// ============================================================
import { HttpErrorResponse } from '@angular/common/http';

export function mensajeError(error: HttpErrorResponse): string {
  const detalle = error.error?.detail;

  if (error.status === 0) {
    return 'No se pudo conectar con la API. Revisa que esté encendida.';
  }
  // 400, 404 y 409: la API ya envía el mensaje en español.
  if (typeof detalle === 'string') {
    return detalle;
  }
  // 422: la API envía una lista de errores. Se muestra el primero.
  if (Array.isArray(detalle)) {
    return detalle[0].msg.replace('Value error, ', '');
  }
  return 'Ocurrió un error en el servidor.';
}
```

Situación	Lo que ve el usuario
La API está apagada (estado 0)	No se pudo conectar con la API. Revisa que esté encendida.
400, 404 o 409	El mensaje que envía la API, por ejemplo: *Ya existe un registro con ese email*
422	El primer error de validación, sin el texto técnico *Value error*
Cualquier otro	Ocurrió un error en el servidor.

11. Modelos y servicios de datos

Crear los archivos

Terminal (carpeta frontend)

```
npx ng g interface models/departamento --type=model
npx ng g interface models/cargo --type=model
npx ng g interface models/empleado --type=model
npx ng g interface models/territorio --type=model
npx ng g service services/api/departamento --type=service --skip-tests
npx ng g service services/api/cargo --type=service --skip-tests
npx ng g service services/api/empleado --type=service --skip-tests
npx ng g service services/api/territorio --type=service --skip-tests
```

Modelos

Un modelo es una **interface** que describe la forma de los datos. TypeScript la usa para avisar si falta un campo o si se escribe mal un nombre.

src/app/models/departamento.model.ts

```
// Datos que se envían a la API al crear o editar.
export interface DepartamentoForm {
  nombre: string;
  descripcion: string;
}

// Departamento completo que devuelve la API.
export interface Departamento extends DepartamentoForm {
  id: number;
}
```

src/app/models/empleado.model.ts

```
// Datos que se envían a la API al crear o editar.
// Los ids parten en null hasta que el usuario elige una opción.
export interface EmpleadoForm {
  rut: string;
  nombres: string;
  apellidos: string;
  email: string;
  telefono: string;
  direccion: string;
  fecha_ingreso: string;
  sueldo: number | null;
  activo: boolean;
  comuna_id: number | null;
  cargo_id: number | null;
  departamento_id: number | null;
}

// Empleado completo que devuelve la API.
export interface Empleado extends EmpleadoForm {
  id: number;
  comuna: string;
  provincia: string;
  region: string;
  cargo: string;
  departamento: string;
}
```

Cada modelo tiene dos interfaces: `EmpleadoForm` con lo que se envía a la API, y `Empleado` con lo que la API devuelve (agrega `id` y los nombres de comuna, cargo y departamento). En `EmpleadoForm` los ids pueden ser `null` porque los selectores parten vacíos.

src/app/models/territorio.model.ts

```
// Forma de los datos territoriales que entrega la API.
export interface Region {
  id: number;
  numero_romano: string;
  nombre: string;
}

export interface Provincia {
  id: number;
  nombre: string;
  region_id: number;
}

export interface Comuna {
  id: number;
  nombre: string;
  provincia_id: number;
  provincia: string;
  region_id: number;
  region: string;
}
```

Servicios de datos

Cada servicio agrupa las rutas de un recurso de la API. Todos usan `ApiService` y devuelven un **Observable**: la petición se envía recién cuando alguien llama a `.subscribe()`.

src/app/services/api/departamento.service.ts

```
// ============================================================
// departamento.service.ts
// Todas las llamadas a la API para departamentos.
// ============================================================
import { Injectable, inject } from '@angular/core';
import { Observable } from 'rxjs';
import { Departamento, DepartamentoForm } from '../../models/departamento.model';
import { ApiService } from './api.service';

@Injectable({ providedIn: 'root' })
export class DepartamentoService {
  private api = inject(ApiService);

  listar(): Observable<Departamento[]> {
    return this.api.get<Departamento[]>('/departamentos');
  }

  crear(datos: DepartamentoForm): Observable<Departamento> {
    return this.api.post<Departamento>('/departamentos', datos);
  }

  actualizar(id: number, datos: DepartamentoForm): Observable<Departamento> {
    return this.api.put<Departamento>(`/departamentos/${id}`, datos);
  }

  eliminar(id: number): Observable<void> {
    return this.api.delete<void>(`/departamentos/${id}`);
  }
}
```

src/app/services/api/empleado.service.ts

```
// ============================================================
// empleado.service.ts
// Todas las llamadas a la API para empleados.
// ============================================================
import { Injectable, inject } from '@angular/core';
import { Observable } from 'rxjs';
import { Empleado, EmpleadoForm } from '../../models/empleado.model';
import { ApiService } from './api.service';

@Injectable({ providedIn: 'root' })
export class EmpleadoService {
  private api = inject(ApiService);

  // La API busca por nombre, apellido o RUT.
  listar(buscar = ''): Observable<Empleado[]> {
    return this.api.get<Empleado[]>('/empleados', { buscar });
  }

  crear(datos: EmpleadoForm): Observable<Empleado> {
    return this.api.post<Empleado>('/empleados', datos);
  }

  actualizar(id: number, datos: EmpleadoForm): Observable<Empleado> {
    return this.api.put<Empleado>(`/empleados/${id}`, datos);
  }

  eliminar(id: number): Observable<void> {
    return this.api.delete<void>(`/empleados/${id}`);
  }
}
```

`listar(buscar)` envía `?buscar=texto`. La búsqueda la hace la API, que ignora tildes y mayúsculas: buscar *nunez* encuentra a *Núñez*.

src/app/services/api/territorio.service.ts

```
// ============================================================
// territorio.service.ts
// Consulta regiones, provincias y comunas en la API.
// ============================================================
import { Injectable, inject } from '@angular/core';
import { Observable } from 'rxjs';
import { Comuna, Provincia, Region } from '../../models/territorio.model';
import { ApiService } from './api.service';

@Injectable({ providedIn: 'root' })
export class TerritorioService {
  private api = inject(ApiService);

  regiones(): Observable<Region[]> {
    return this.api.get<Region[]>('/regiones');
  }

  provincias(regionId: number): Observable<Provincia[]> {
    return this.api.get<Provincia[]>(`/regiones/${regionId}/provincias`);
  }

  comunas(provinciaId: number): Observable<Comuna[]> {
    return this.api.get<Comuna[]>(`/provincias/${provinciaId}/comunas`);
  }

  comuna(id: number): Observable<Comuna> {
    return this.api.get<Comuna>(`/comunas/${id}`);
```

```
  }
}
```

`cargo.service.ts` es igual a `departamento.service.ts`, pero con la ruta `/cargos`.

12. Etapa 4: Páginas del mantenedor

Un **mantenedor** es una pantalla para administrar datos: listar, buscar, crear, editar y eliminar. Las tres páginas siguen el mismo patrón:

1. Al abrirse (`ngOnInit`), piden la lista a la API.
2. El formulario está enlazado con `[(ngModel)]`: lo que se escribe queda en `this.form`.
3. Si `editandoId` es `null`, Guardar crea; si tiene un número, actualiza ese registro.
4. Después de guardar o eliminar se muestra una alerta, se registra en el log y se recarga la lista.
5. Si algo falla, se muestra el error con `mensajeError()`.

Crear los archivos

Terminal (carpeta frontend)

```
npx ng g component components/menu --skip-tests
npx ng g component pages/departamentos --skip-tests --style=none
npx ng g component pages/cargos --skip-tests --style=none
npx ng g component pages/empleados --skip-tests --style=none
```

`ng generate` no agrega las páginas a las rutas: hay que registrarlas en `app.routes.ts` (sección 6). El menú, las alertas y la carga se agregan en `app.html`.

El menú

src/app/components/menu/menu.html

```
<header>
  <strong>Control de Empleados</strong>
  <nav>
    <a routerLink="/empleados" routerLinkActive="activo">Empleados</a>
    <a routerLink="/departamentos" routerLinkActive="activo">Departamentos</a>
    <a routerLink="/cargos" routerLinkActive="activo">Cargos</a>
    <a routerLink="/logs" routerLinkActive="activo">Registro de eventos</a>
  </nav>
</header>
```

`routerLink` cambia de página sin recargar el navegador. `routerLinkActive="activo"` agrega la clase `activo` al enlace de la página actual, para subrayarlo.

Departamentos

src/app/pages/departamentos/departamentos.ts

```
// ============================================================
// departamentos.ts
// Mantenedor de departamentos: listar, buscar, crear, editar y eliminar.
// ============================================================
import { HttpErrorResponse } from '@angular/common/http';
import { Component, OnInit, computed, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Departamento, DepartamentoForm } from '../../models/departamento.model';
import { mensajeError } from '../../services/api/mensaje-error';
import { AlertaService } from '../../services/ui/alerta.service';
import { DepartamentoService } from '../../services/api/departamento.service';
```

```
import { LogService } from '../../services/sistema/log.service';

@Component({
  selector: 'app-departamentos',
  imports: [FormsModule],
  templateUrl: './departamentos.html',
})
export class Departamentos implements OnInit {
  private servicio = inject(DepartamentoService);
  private alerta = inject(AlertaService);
  private log = inject(LogService);

  lista = signal<Departamento[]>([]);
  buscar = signal('');
  editandoId = signal<number | null>(null); // null = creando

  // Datos del formulario, enlazados con [(ngModel)].
  form: DepartamentoForm = { nombre: '', descripcion: '' };

  // computed se recalcula solo cuando cambia la lista o la búsqueda.
  filtrados = computed(() => {
    const texto = this.buscar().toLowerCase();
    return this.lista().filter((item) => item.nombre.toLowerCase().includes(texto));
  });

  ngOnInit(): void {
    this.cargar();
  }

  cargar(): void {
    this.servicio.listar().subscribe({
      next: (datos) => this.lista.set(datos),
      error: (error: HttpErrorResponse) => this.alerta.error(mensajeError(error)),
    });
  }

  guardar(): void {
    const id = this.editandoId();
    const peticion = id ? this.servicio.actualizar(id, this.form) : this.servicio.crear(this.form);

    peticion.subscribe({
      next: () => {
        this.alerta.exito('Departamento guardado correctamente.');
        this.log.info(`Departamento guardado: ${this.form.nombre}`);
        this.limpiar();
        this.cargar();
      },
      error: (error: HttpErrorResponse) => this.alerta.error(mensajeError(error)),
    });
  }

  // Copia los datos de la fila al formulario.
  editar(item: Departamento): void {
    this.form = { nombre: item.nombre, descripcion: item.descripcion };
    this.editandoId.set(item.id);
  }

  eliminar(item: Departamento): void {
    if (!confirm(`¿Eliminar "${item.nombre}"?`)) {
      return;
    }
    this.servicio.eliminar(item.id).subscribe({
      next: () => {
        this.alerta.exito('Departamento eliminado.');
        this.log.info(`Departamento eliminado: ${item.nombre}`);
        this.cargar();
      },
      error: (error: HttpErrorResponse) => this.alerta.error(mensajeError(error)),
```

```
    });
  }

  limpiar(): void {
    this.form = { nombre: '', descripcion: '' };
    this.editandoId.set(null);
  }
}
```

src/app/pages/departamentos/departamentos.html

```
<h1>Departamentos</h1>

<form class="formulario" #f="ngForm" (ngSubmit)="guardar()">
  <h2>{{ editandoId() ? 'Editar' : 'Nuevo' }}</h2>

  <label>
    Nombre
    <input name="nombre" [(ngModel)]="form.nombre" required />
  </label>
  <label>
    Descripción
    <input name="descripcion" [(ngModel)]="form.descripcion" />
  </label>

  <div class="acciones">
    <button type="submit" class="principal" [disabled]="f.invalid">Guardar</button>
    <button type="button" (click)="limpiar()">Cancelar</button>
  </div>
</form>

<input class="buscador" placeholder="Buscar por nombre" name="buscar" [(ngModel)]="buscar" />

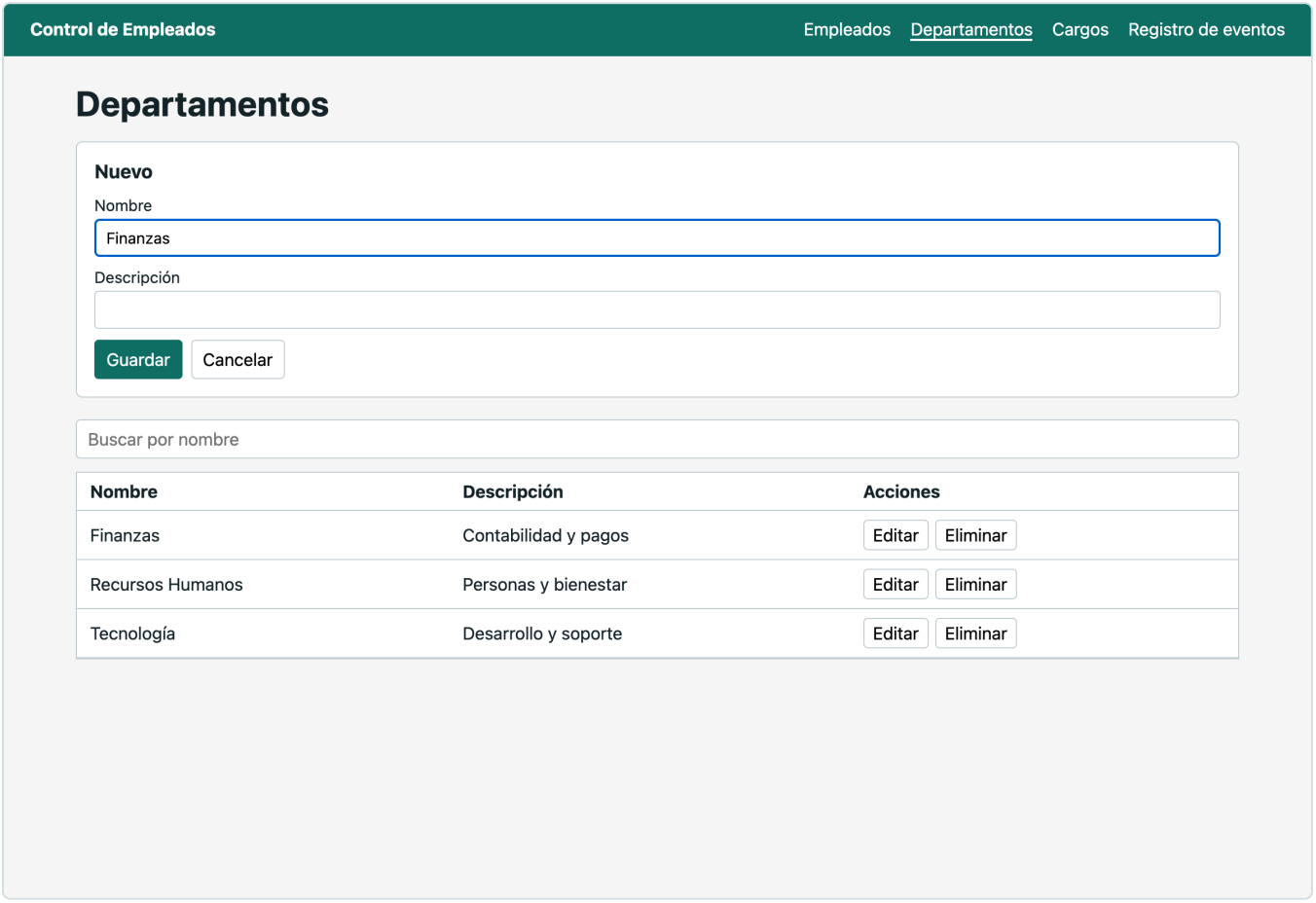
<div class="tabla">
  <table>
    <thead>
      <tr>
        <th>Nombre</th>
        <th>Descripción</th>
        <th>Acciones</th>
      </tr>
    </thead>
    <tbody>
      @for (item of filtrados(); track item.id) {
        <tr>
          <td>{{ item.nombre }}</td>
          <td>{{ item.descripcion }}</td>
          <td>
            <button (click)="editar(item)">Editar</button>
            <button (click)="eliminar(item)">Eliminar</button>
          </td>
        </tr>
      } @empty {
        <tr>
          <td colspan="3">No hay registros.</td>
        </tr>
      }
    </tbody>
  </table>
</div>
```

Puntos importantes

- `#f="ngForm"` da acceso al formulario completo. `[disabled]="f.invalid"` desactiva Guardar mientras falte un campo `required`.

- `filtrados` es un `computed`: filtra la lista mientras se escribe en el buscador, sin pedir nada a la API.
- `confirm()` pregunta antes de eliminar.
- Si el departamento tiene empleados, la API responde 409 y aparece la alerta *El registro tiene empleados asociados*.

La página **Cargos** es idéntica, con `CargoService` en lugar de `DepartamentoService`.



Mantenedor de departamentos.

Empleados

Es la página más completa: tiene más campos, la búsqueda se hace en la API y los selectores de región, provincia y comuna van en cascada.

src/app/pages/empleados/empleados.ts

```
// ============================================================
// empleados.ts
// Mantenedor de empleados: listar, buscar, crear, editar y eliminar.
// Los selectores van en cascada: región -> provincia -> comuna.
// ============================================================
import { HttpErrorResponse } from '@angular/common/http';
import { Component, OnInit, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Cargo } from '../../models/cargo.model';
import { Departamento } from '../../models/departamento.model';
import { Empleado, EmpleadoForm } from '../../models/empleado.model';
import { Comuna, Provincia, Region } from '../../models/territorio.model';
import { mensajeError } from '../../services/api/mensaje-error';
import { AlertaService } from '../../services/ui/alerta.service';
import { CargoService } from '../../services/api/cargo.service';
```

```
import { DepartamentoService } from '../../services/api/departamento.service';
import { EmpleadoService } from '../../services/api/empleado.service';
import { LogService } from '../../services/sistema/log.service';
import { TerritorioService } from '../../services/api/territorio.service';

@Component({
  selector: 'app-empleados',
  imports: [FormsModule],
  templateUrl: './empleados.html',
})
export class Empleados implements OnInit {
  private empleadoService = inject(EmpleadoService);
  private departamentoService = inject(DepartamentoService);
  private cargoService = inject(CargoService);
  private territorio = inject(TerritorioService);
  private alerta = inject(AlertaService);
  private log = inject(LogService);

  // Datos que se muestran en la tabla y en los selectores.
  empleados = signal<Empleado[]>([]);
  departamentos = signal<Departamento[]>([]);
  cargos = signal<Cargo[]>([]);
  regiones = signal<Region[]>([]);
  provincias = signal<Provincia[]>([]);
  comunas = signal<Comuna[]>([]);

  editandoId = signal<number | null>(null); // null = creando
  buscar = '';

  // Datos del formulario, enlazados con [(ngModel)].
  form: EmpleadoForm = this.formVacio();
  // Solo sirven para los selectores: al guardar basta con comuna_id.
  regionId: number | null = null;
  provinciaId: number | null = null;

  // Se usa en todas las peticiones de esta página.
  mostrarError = (error: HttpErrorResponse) => this.alerta.error(mensajeError(error));

  ngOnInit(): void {
    this.cargar();
    this.departamentoService
      .listar()
      .subscribe({ next: (datos) => this.departamentos.set(datos), error: this.mostrarError });
    this.cargoService
      .listar()
      .subscribe({ next: (datos) => this.cargos.set(datos), error: this.mostrarError });
    this.territorio
      .regiones()
      .subscribe({ next: (datos) => this.regiones.set(datos), error: this.mostrarError });
  }

  cargar(): void {
    this.empleadoService.listar(this.buscar).subscribe({
      next: (datos) => this.empleados.set(datos),
      error: this.mostrarError,
    });
  }

  // Al cambiar la región se cargan sus provincias y se borra lo elegido abajo.
  cambiarRegion(id: number | null): void {
    this.regionId = id;
    this.provinciaId = null;
    this.form.comuna_id = null;
    this.provincias.set([]);
    this.comunas.set([]);
    if (id) {
      this.territorio
        .provincias(id)
```

```
        .subscribe({ next: (datos) => this.provincias.set(datos), error: this.mostrarError });
    }
  }

  // Al cambiar la provincia se cargan sus comunas.
  cambiarProvincia(id: number | null): void {
    this.provinciaId = id;
    this.form.comuna_id = null;
    this.comunas.set([]);
    if (id) {
      this.territorio
        .comunas(id)
        .subscribe({ next: (datos) => this.comunas.set(datos), error: this.mostrarError });
    }
  }

  guardar(): void {
    const id = this.editandoId();
    const peticion = id
      ? this.empleadoService.actualizar(id, this.form)
      : this.empleadoService.crear(this.form);

    peticion.subscribe({
      next: () => {
        this.alerta.exito('Empleado guardado correctamente.');
        this.log.info(`Empleado guardado: ${this.form.nombres} ${this.form.apellidos}`);
        this.limpiar();
        this.cargar();
      },
      error: this.mostrarError,
    });
  }

  // Copia los datos de la fila al formulario.
  editar(empleado: Empleado): void {
    this.form = {
      rut: empleado.rut,
      nombres: empleado.nombres,
      apellidos: empleado.apellidos,
      email: empleado.email,
      telefono: empleado.telefono,
      direccion: empleado.direccion,
      fecha_ingreso: empleado.fecha_ingreso,
      sueldo: empleado.sueldo,
      activo: empleado.activo,
      comuna_id: empleado.comuna_id,
      cargo_id: empleado.cargo_id,
      departamento_id: empleado.departamento_id,
    };
    this.editandoId.set(empleado.id);

    // La API indica la región y provincia de la comuna, para llenar los selectores.
    this.territorio.comuna(empleado.comuna_id!).subscribe({
      next: (comuna) => {
        this.regionId = comuna.region_id;
        this.provinciaId = comuna.provincia_id;
        this.territorio
          .provincias(comuna.region_id)
          .subscribe({ next: (datos) => this.provincias.set(datos), error: this.mostrarError });
        this.territorio
          .comunas(comuna.provincia_id)
          .subscribe({ next: (datos) => this.comunas.set(datos), error: this.mostrarError });
      },
      error: this.mostrarError,
    });
  }

  eliminar(empleado: Empleado): void {
```

```
    if (!confirm(`¿Eliminar a ${empleado.nombres} ${empleado.apellidos}?`)) {
      return;
    }
    this.empleadoService.eliminar(empleado.id).subscribe({
      next: () => {
        this.alerta.exito('Empleado eliminado.');
        this.log.info(`Empleado eliminado: ${empleado.nombres} ${empleado.apellidos}`);
        this.cargar();
      },
      error: this.mostrarError,
    });
  }

  limpiar(): void {
    this.form = this.formVacio();
    this.editandoId.set(null);
    this.regionId = null;
    this.provinciaId = null;
    this.provincias.set([]);
    this.comunas.set([]);
  }

  private formVacio(): EmpleadoForm {
    return {
      rut: '',
      nombres: '',
      apellidos: '',
      email: '',
      telefono: '',
      direccion: '',
      fecha_ingreso: '',
      sueldo: null,
      activo: true,
      comuna_id: null,
      cargo_id: null,
      departamento_id: null,
    };
  }
}
```

src/app/pages/empleados/empleados.html

```
<h1>Empleados</h1>

<form class="formulario" #f="ngForm" (ngSubmit)="guardar()">
  <h2>{{ editandoId() ? 'Editar empleado' : 'Nuevo empleado' }}</h2>

  <div class="campos">
    <label>
      RUT
      <input name="rut" placeholder="12345678-5" [(ngModel)]="form.rut" required />
    </label>
    <label>
      Nombres
      <input name="nombres" [(ngModel)]="form.nombres" required />
    </label>
    <label>
      Apellidos
      <input name="apellidos" [(ngModel)]="form.apellidos" required />
    </label>
    <label>
      Correo
      <input type="email" name="email" [(ngModel)]="form.email" required email />
    </label>
    <label>
      Teléfono
      <input name="telefono" [(ngModel)]="form.telefono" />
```

```
    </label>
    <label>
      Dirección
      <input name="direccion" [(ngModel)]="form.direccion" />
    </label>
    <label>
      Fecha de ingreso
      <input type="date" name="fecha_ingreso" [(ngModel)]="form.fecha_ingreso" required />
    </label>
    <label>
      Sueldo
      <input type="number" name="sueldo" min="0" [(ngModel)]="form.sueldo" required />
    </label>

    <label>
      Departamento
      <select name="departamento_id" [(ngModel)]="form.departamento_id" required>
        <option [ngValue]="null">Seleccione</option>
        @for (departamento of departamentos(); track departamento.id) {
          <option [ngValue]="departamento.id">{{ departamento.nombre }}</option>
        }
      </select>
    </label>
    <label>
      Cargo
      <select name="cargo_id" [(ngModel)]="form.cargo_id" required>
        <option [ngValue]="null">Seleccione</option>
        @for (cargo of cargos(); track cargo.id) {
          <option [ngValue]="cargo.id">{{ cargo.nombre }}</option>
        }
      </select>
    </label>

    <label>
      Región
      <select name="region" [ngModel]="regionId" (ngModelChange)="cambiarRegion($event)" required>
        <option [ngValue]="null">Seleccione</option>
        @for (region of regiones(); track region.id) {
          <option [ngValue]="region.id">{{ region.nombre }}</option>
        }
      </select>
    </label>
    <label>
      Provincia
      <select
        name="provincia"
        [ngModel]="provinciaId"
        (ngModelChange)="cambiarProvincia($event)"
        required
      >
        <option [ngValue]="null">Seleccione</option>
        @for (provincia of provincias(); track provincia.id) {
          <option [ngValue]="provincia.id">{{ provincia.nombre }}</option>
        }
      </select>
    </label>
    <label>
      Comuna
      <select name="comuna_id" [(ngModel)]="form.comuna_id" required>
        <option [ngValue]="null">Seleccione</option>
        @for (comuna of comunas(); track comuna.id) {
          <option [ngValue]="comuna.id">{{ comuna.nombre }}</option>
        }
      </select>
    </label>

    <label class="check">
      <input type="checkbox" name="activo" [(ngModel)]="form.activo" />
```

```
      Activo
    </label>
  </div>

  <div class="acciones">
    <button type="submit" class="principal" [disabled]="f.invalid">Guardar</button>
    <button type="button" (click)="limpiar()">Cancelar</button>
  </div>
</form>

<form class="busqueda" (ngSubmit)="cargar()">
  <input placeholder="Buscar por nombre o RUT" name="buscar" [(ngModel)]="buscar" />
  <button type="submit">Buscar</button>
</form>

<div class="tabla">
  <table>
    <thead>
      <tr>
        <th>RUT</th>
        <th>Nombre</th>
        <th>Departamento</th>
        <th>Cargo</th>
        <th>Comuna</th>
        <th>Sueldo</th>
        <th>Estado</th>
        <th>Acciones</th>
      </tr>
    </thead>
    <tbody>
      @for (empleado of empleados(); track empleado.id) {
        <tr>
          <td>{{ empleado.rut }}</td>
          <td>{{ empleado.nombres }} {{ empleado.apellidos }}</td>
          <td>{{ empleado.departamento }}</td>
          <td>{{ empleado.cargo }}</td>
          <td>{{ empleado.comuna }}</td>
          <td>$ {{ empleado.sueldo?.toLocaleString('es-CL') }}</td>
          <td>{{ empleado.activo ? 'Activo' : 'Inactivo' }}</td>
          <td>
            <button (click)="editar(empleado)">Editar</button>
            <button (click)="eliminar(empleado)">Eliminar</button>
          </td>
        </tr>
      } @empty {
        <tr>
          <td colspan="8">No hay empleados.</td>
        </tr>
      }
    </tbody>
  </table>
</div>
```

Selectores en cascada

Las comunas dependen de la provincia, y las provincias dependen de la región:

1. Al abrir la página se cargan las 16 regiones.
2. Al elegir una región, `cambiarRegion()` limpia la provincia y la comuna, y pide las provincias de esa región.
3. Al elegir una provincia, `cambiarProvincia()` limpia la comuna y pide sus comunas.
4. Solo `comuna_id` se envía a la API: la región y la provincia se deducen de la comuna.

En el HTML se usa `[ngModel]` con `(ngModelChange)` en vez de `[(ngModel)]`, porque al cambiar el valor hay que hacer algo más: cargar la lista siguiente.

Editar un empleado

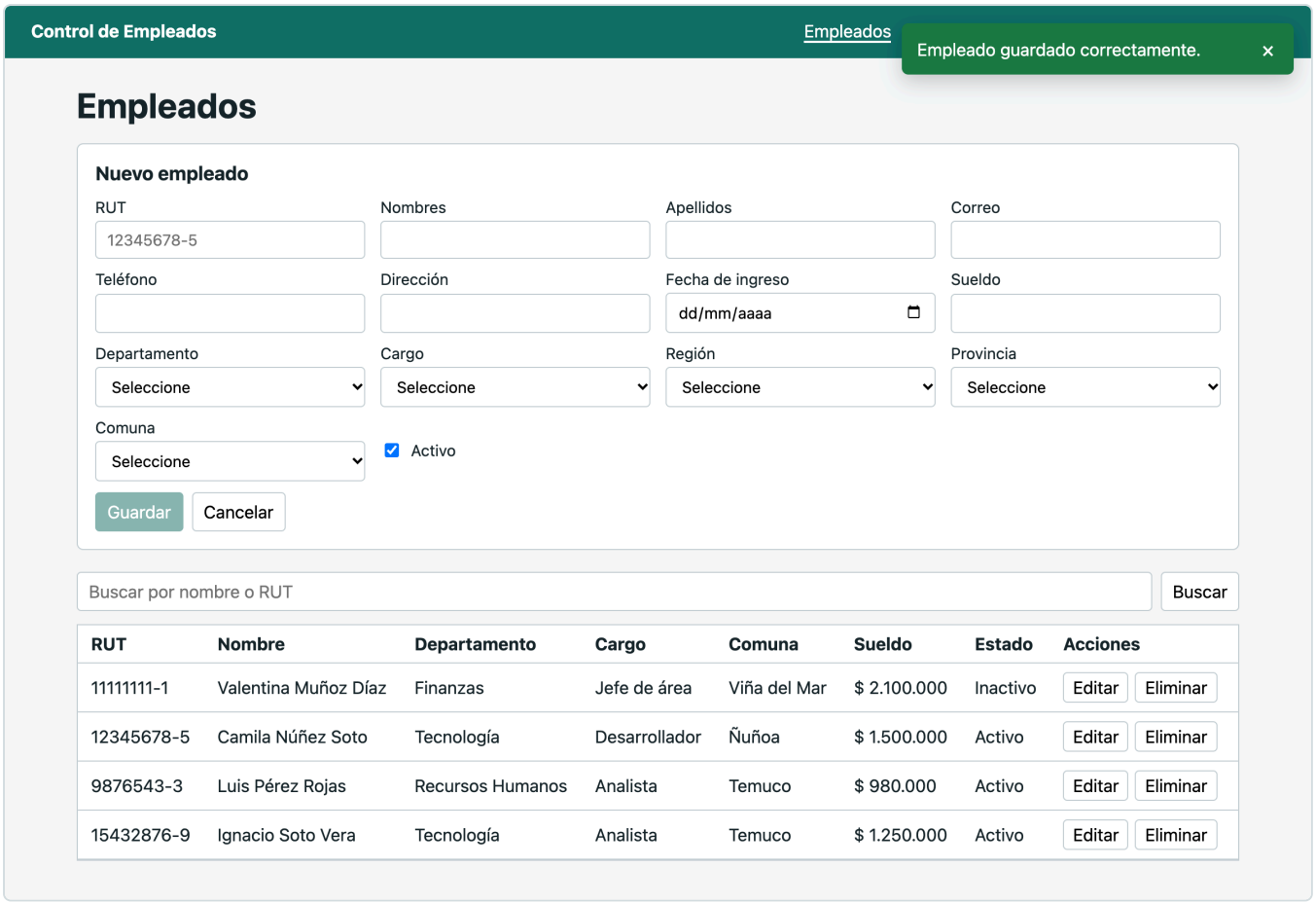
El empleado guarda solo `comuna_id`. Para llenar los selectores, `editar()` consulta `GET /comunas/{id}`, que devuelve la región y la provincia de esa comuna, y con eso carga las listas.

Manejo de errores

`mostrarError` se define una vez y se usa en todas las peticiones de la página. Así, si la API está apagada, el usuario ve un mensaje claro en vez de un error inesperado.



Formulario de empleado con los selectores en cascada completos.



Empleado guardado: aparece la alerta de éxito y el nuevo registro en la tabla.

Estilos comunes

Los estilos de formularios, botones y tablas están una sola vez en `styles.css`, y todas las páginas los comparten.

src/styles.css

```
/* Estilos que comparten todas las páginas. */
:root {
  --primario: #0f6f68;
  --borde: #c6d1d6;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #f5f7f8;
  color: #17242b;
  font-family: system-ui, sans-serif;
}

.contenido {
  max-width: 1100px;
  margin: 0 auto;
  padding: 24px 16px;
}

h1 {
  margin: 0 0 16px;
```

```
}

h2 {
  margin: 0 0 12px;
  font-size: 1.1rem;
}

input,
select,
button {
  padding: 8px 10px;
  border: 1px solid var(--borde);
  border-radius: 4px;
  background: #fff;
  font: inherit;
}

button {
  cursor: pointer;
}

button.principal {
  background: var(--primario);
  border-color: var(--primario);
  color: #fff;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* Formularios */
.formulario {
  margin-bottom: 20px;
  padding: 16px;
  background: #fff;
  border: 1px solid var(--borde);
  border-radius: 6px;
}

.formulario label {
  display: grid;
  gap: 4px;
  margin-bottom: 10px;
  font-size: 0.9rem;
}

.formulario input:not([type='checkbox']),
.formulario select {
  width: 100%;
  min-width: 0;
}

.campos {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 0 14px;
}

.formulario label.check {
  display: flex;
  align-items: center;
  gap: 8px;
}

.acciones,
.busqueda {
```

```
  display: flex;
  gap: 8px;
}

.buscador,
.busqueda input {
  flex: 1;
  width: 100%;
}

.buscador,
.busqueda {
  margin-bottom: 12px;
}

/* Tablas: en pantallas chicas se desplazan dentro de su caja. */
.tabla {
  overflow-x: auto;
  background: #fff;
  border: 1px solid var(--borde);
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  padding: 8px 12px;
  border-bottom: 1px solid var(--borde);
  text-align: left;
  white-space: nowrap;
}

td button {
  margin-right: 6px;
  padding: 4px 8px;
}
```

13. Pruebas y resultados

Pruebas unitarias

Las pruebas unitarias revisan cada servicio por separado, sin abrir el navegador. Angular 22 usa **Vitest**. Cada archivo `.spec.ts` está junto al servicio que prueba.

src/app/services/ui/carga.service.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { describe, expect, it } from 'vitest';
import { CargaService } from './carga.service';

describe('CargaService', () => {
  it('sigue activa hasta que termina la última petición', () => {
    const carga = TestBed.inject(CargaService);
    expect(carga.activa()).toBe(false);
    carga.mostrar();
    carga.mostrar();
    carga.ocultar();
    expect(carga.activa()).toBe(true);
    carga.ocultar();
    expect(carga.activa()).toBe(false);
  });
});
```

src/app/services/api/api.service.spec.ts

```
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
import { TestBed } from '@angular/core/testing';
import { beforeEach, describe, expect, it, vi } from 'vitest';
import { environment } from '../../../environments/environment';
import { LogService } from '../sistema/log.service';
import { CargaService } from '../ui/carga.service';
import { ApiService } from './api.service';

describe('ApiService', () => {
  const url = `${environment.apiUrl}/cargos`;
  let api: ApiService;
  let servidor: HttpTestingController;
  let carga: CargaService;
  let log: LogService;

  beforeEach(() => {
    vi.spyOn(console, 'log').mockImplementation(() => {});
    TestBed.configureTestingModule({
      providers: [provideHttpClient(), provideHttpClientTesting()],
    });
    api = TestBed.inject(ApiService);
    servidor = TestBed.inject(HttpTestingController);
    carga = TestBed.inject(CargaService);
    log = TestBed.inject(LogService);
  });

  it('activa la carga y registra el log cuando sale bien', () => {
    api.get('/cargos').subscribe();
    expect(carga.activa()).toBe(true);
    servidor.expectOne(url).flush([]);
    expect(carga.activa()).toBe(false);
    expect(log.logs()[0].mensaje).toBe('GET /cargos: correcto');
  });
```

```
  it('oculta la carga y registra el error cuando falla', () => {
    api.get('/cargos').subscribe({ error: () => {} });
    servidor.expectOne(url).flush(null, { status: 500, statusText: 'Error' });
    expect(carga.activa()).toBe(false);
    expect(log.logs()[0].nivel).toBe('error');
  });
});
```

`HttpTestingController` reemplaza a la API real: la prueba decide qué responde (`flush`) y revisa qué hizo el servicio.

Terminal

```
npx ng test --watch=false --reporters=verbose
```

Resultado obtenido

```
RUN  v5.0.5 frontend

 ✓  frontend  src/app/services/api/mensaje-error.spec.ts > mensajeError > usa el mensaje que envía la API 1ms
 ✓  frontend  src/app/services/api/mensaje-error.spec.ts > mensajeError > limpia los errores de validación 0ms
 ✓  frontend  src/app/services/api/mensaje-error.spec.ts > mensajeError > avisa cuando la API no responde 0ms
 ✓  frontend  src/app/services/ui/alerta.service.spec.ts > AlertaService > se cierra sola después de 5 segundos 9ms
 ✓  frontend  src/app/services/ui/alerta.service.spec.ts > AlertaService > no repite un mensaje que ya está en pantalla 2ms
 ✓  frontend  src/app/services/ui/carga.service.spec.ts > CargaService > sigue activa hasta que termina la última petición 8ms
 ✓  frontend  src/app/services/sistema/log.service.spec.ts > LogService > guarda los registros con el más nuevo primero 10ms
 ✓  frontend  src/app/services/sistema/log.service.spec.ts > LogService > limpia los registros 2ms
 ✓  frontend  src/app/services/api/api.service.spec.ts > ApiService > activa la carga y registra el log cuando sale bien 11ms
 ✓  frontend  src/app/services/api/api.service.spec.ts > ApiService > oculta la carga y registra el error cuando falla 2ms

 Test Files  5 passed (5)
      Tests  10 passed (10)
   Start at  08:31:50
   Duration  1.03s (environment 75%, setup 18%, transform 5%, tests 1%)
```

Compilación

Terminal

```
npx ng build
```

Resultado obtenido

```
Initial chunk files | Names         |  Raw size | Estimated transfer size
main-VYQFAKZO.js    | main          | 340.78 kB |                88.08 kB
styles-ZSHZBNXD.css | styles        |   1.32 kB |               484 bytes

                    | Initial total | 342.10 kB |                88.56 kB

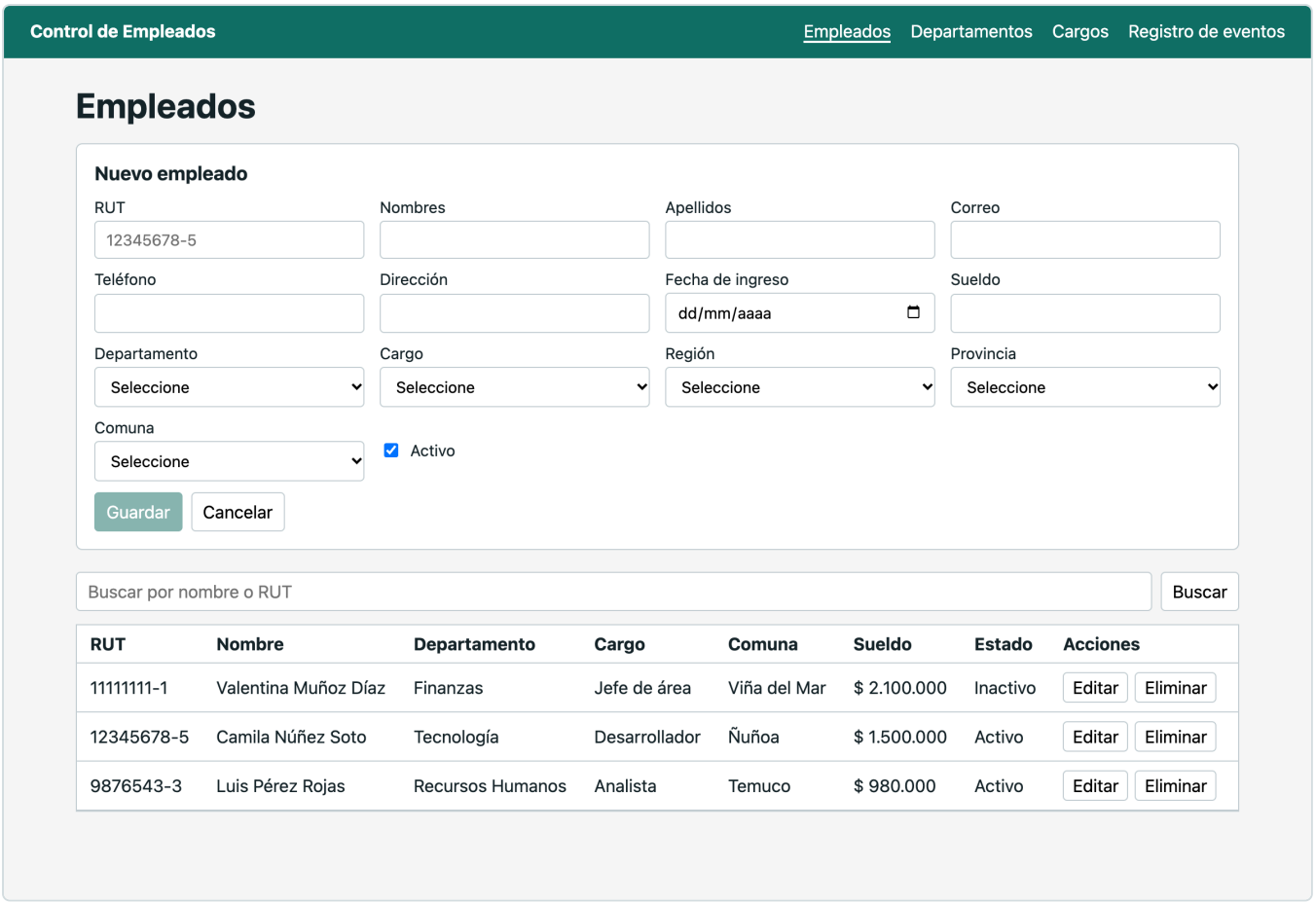
Application bundle generation complete. [1.449 seconds] - 2026-11-02T11:31:53.608Z

Output location: frontend/dist/frontend
```

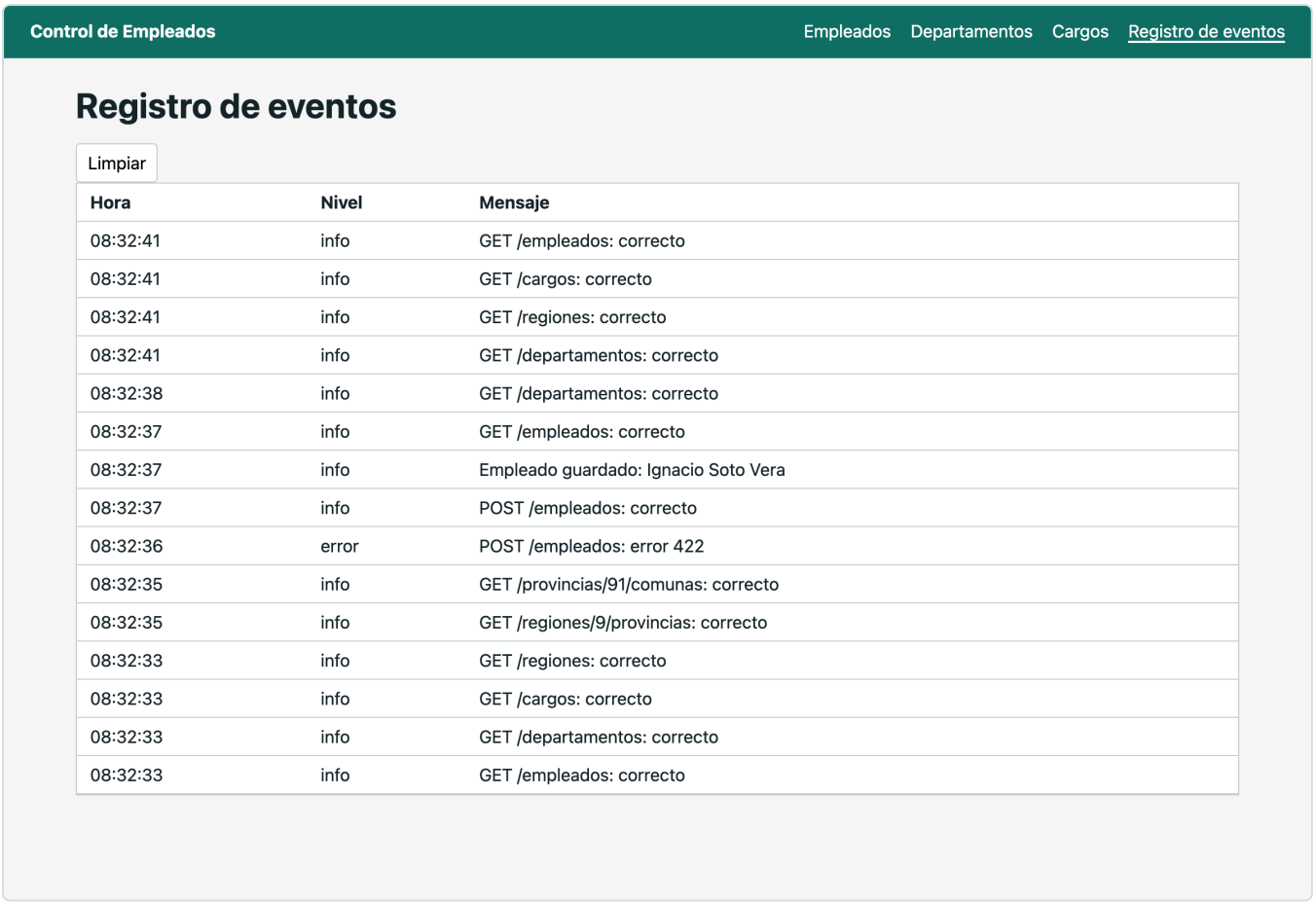
Pruebas en el navegador

Además, la aplicación se probó en Chrome con la API encendida. Estas pruebas se hicieron contra una base de datos temporal, para no modificar la base real.

Prueba	Resultado esperado	Resultado
Crear, editar, buscar y eliminar departamentos	La tabla se actualiza y aparece la alerta de éxito	Correcto
Crear un departamento con nombre repetido	Alerta: Ya existe un registro con ese nombre	Correcto
Crear, editar y eliminar cargos	Igual que departamentos	Correcto
Elegir región, provincia y comuna	Cada lista se llena según la anterior	Correcto
Cambiar la región al editar	Provincia y comuna se limpian y Guardar se desactiva	Correcto
Guardar un RUT con dígito verificador incorrecto	Alerta de la API (422)	Correcto
Guardar un correo repetido	Alerta: Ya existe un registro con ese email	Correcto
Buscar “nunez” y buscar por RUT	Encuentra a Núñez y al RUT buscado	Correcto
Marcar un empleado como inactivo	La tabla muestra Inactivo	Correcto
Eliminar un departamento con empleados	Alerta: El registro tiene empleados asociados	Correcto
Petición lenta (1,5 s simulada)	Pantalla oscura, sin clics ni teclado	Correcto
API apagada	Una sola alerta: No se pudo conectar con la API	Correcto
Pantalla de celular (390 px)	Sin desplazamiento horizontal	Correcto



Página de empleados con datos de ejemplo.



Registro de eventos: cada petición queda anotada, incluido el error 422 del RUT inválido.



Vista en un celular: el formulario pasa a una columna.

14. Problemas comunes

Síntoma	Causa	Solución
Alerta “No se pudo conectar con la API”	El backend no está encendido	Ejecutar `uvicorn main:app --reload` en la carpeta backend
En la consola (F12): “blocked by CORS policy”	Angular corre en otro puerto que no es 4200	Usar `npm start` (puerto 4200) o agregar el puerto en `allow_origins` del backend
NullInjectorError: No provider for HttpClient	Falta `provideHttpClient()`	Agregarlo en `app.config.ts`
Can't bind to 'ngModel'	El componente no importa `FormsModule`	Agregar `FormsModule` en `imports` del componente
El selector de empleados no muestra departamentos	Todavía no hay departamentos creados	Crear primero un departamento y un cargo
Port 4200 is already in use	Ya hay otro `npm start` corriendo	Cerrar la otra terminal o usar `npm start -- --port 4201` (y agregar ese puerto al CORS)
El spinner no aparece en una petición nueva	El servicio usa `HttpClient` directo	Hacer la petición con `ApiService`

15. Cómo seguir ampliando el proyecto

Para agregar un mantenedor nuevo, por ejemplo **Sucursales**, se repiten los mismos pasos:

1. Crear la tabla y los endpoints en el backend.
2. Crear el modelo y escribir en él `SucursalForm` y `Sucursal`.
3. Crear el servicio y escribir sus métodos usando `ApiService`.
4. Crear la página y completarla copiando el patrón de Departamentos.
5. Agregar la ruta en `app.routes.ts` y el enlace en `components/menu/menu.html`.

Terminal (carpeta frontend)

```
npx ng g interface models/sucursal --type=model
npx ng g service services/api/sucursal --type=service --skip-tests
npx ng g component pages/sucursales --skip-tests --style=none
```

Al usar `ApiService`, `AlertaService` y `LogService`, la página nueva tiene desde el comienzo pantalla de carga, alertas y registro de eventos.

Ideas para practicar

- Agregar en Empleados un filtro por departamento (la API ya acepta `?departamento_id=`).
- Mostrar el total de empleados activos sobre la tabla, usando un `computed`.
- Escribir una prueba unitaria para `TerritorioService`.
- Validar el RUT en el formulario antes de enviarlo a la API.